25900312 **Reg.No:.....**

Name :.....

MAHATMA GANDHI UNIVERSITY, KOTTAYAM
MGU-UGP (HONOURS) Regular EXAMINATION October 2025
Third SEMESTER
Discipline Specific Core Course (DSC) - MG3DSCCMA201 - WEB DEVELOPMENT USING PHP
(2024 ADMISSION ONWARDS)

Duration: 1 Hour 30 Minutes **Maximum Marks: 50**

ANSWER KEY

Remember(K), Understand(U), Apply(A), Analyse(An), Evaluate(E), Create(C), Skill(S), Interest(I) and Appreciation(Ap)

Students should attempt at least one question from each course outcome to enhance their overall outcome attainability.

Part A

Very Short Answer Questions
Answer all questions
Each question carries **2** marks

1. Recall how the if-else statement differs from a simple if statement in PHP? **[U] / [CO1]**
Solution : if executes a block of code only if the condition is true.
if-else provides an alternative block of code to execute if the condition is false.

2. Which is more secure – Cookie or Session? [U] / [CO2]
Solution: Session is more secure because session data is stored on the server, while cookies are stored on the client-side and can be manipulated

3. Write the SQL query to create a table named student. [U] / [CO3]
Solution :

```
CREATE TABLE student
(
 id INT,
 name VARCHAR(100),
 age INT );
```

4. Identify the default port used by Apache. [U] / [CO1]
Solution : Port 80 (HTTP) or Port 443 (HTTPS)

5. Write the SQL command to display all tables in a database [U] / [CO3]
Solution : SHOW TABLES;

[2x5 = 10]

Part B

Short Answer Questions
Answer any **5** questions
Each question carries **5** marks

6. Demonstrate the use of logical operator in decision making by using suitable example [U] / [CO1]

Solution:

Logical operators in PHP are crucial for combining conditional statements in decision-making processes. They allow you to test multiple conditions simultaneously within structures like if, else if, and while loops.

Operator	Name	Example Condition
&&	AND	A && B Returns TRUE if both A and B are TRUE.
||	OR	A || B Returns TRUE if either A or B (or both) is TRUE.
!	NOT	!A Returns the opposite of A. If A is TRUE, !A is FALSE

```
<?php
// Variables
$userAge = 20;
$isRegistered = true;
$isCitizen = false;
```

AND (&&) Checks if user is at least 18 **and** registered:
if ($userAge >= 18 && $isRegistered)
Output: "Access granted! Eligible to Vote."

OR (||) Checks if user is either registered **or** a citizen:
if ($isRegistered || $isCitizen)
Output: "You have a valid status."

NOT (!) Checks if user is **not** a citizen:
if (!($isCitizen))
Output: "User is NOT a citizen."

7. Explain the difference between traditional error handling (die(), error_reporting) and exception handling in PHP [U] / [CO2]

Solution:

Traditional Error Handling

- **Passive approach** using PHP's internal mechanisms.
- Relies on functions like error_reporting(), die(), and set_error_handler().
- Treats issues as non-recoverable events (e.g., warnings, notices, fatal errors).
- Fatal errors abruptly stop execution.
- Limited flexibility and control over error types.

Exception Handling

- **Active, object-oriented approach** for managing recoverable issues.
- Uses try, catch, throw, and optionally finally.
- Allows graceful recovery and structured error management.
- Control jumps to the nearest catch block when an exception is thrown.
- Highly flexible — supports custom exception classes.

Feature Comparison

Feature	Traditional Error Handling	Exception Handling
Concept	Errors (internal events)	Exceptions (objects)
Control Flow	Abrupt stop or ignored warnings	Jumps to catch block for recovery
Mechanism	error_reporting(), die()	try, catch, throw, finally
Flexibility	Low	High — supports custom exception classes
Best Used For	Basic debugging, non-recoverable faults	Professional apps needing structured control

8. Write an SQL statement to create a table named student with fields id, name, and age. [U] / [CO3]
Solution:

```
CREATE TABLE student (
 id INT PRIMARY KEY,
 name VARCHAR(100),
 age INT );
```

9. Explain error handling during file uploads in PHP. [U] / [CO1]
Solution :

1. The $_FILES Array : When a file is uploaded, PHP populates $_FILES['file'] with:

- name: Original filename
- type: MIME type
- size: File size in bytes
- tmp_name: Temporary server path
- error: Numeric upload status code

2. Standard PHP Upload Error Codes

Constant	Value	Description	Action
UPLOAD_ERR_OK	0	Upload successful	Proceed with processing
UPLOAD_ERR_INI_SIZE	1	File exceeds php.ini size limit	Notify user
UPLOAD_ERR_FORM_SIZE	2	File exceeds HTML form size limit	Notify user
UPLOAD_ERR_PARTIAL	3	File partially uploaded	Ask user to retry
UPLOAD_ERR_NO_FILE	4	No file uploaded	Prompt user to select a file
UPLOAD_ERR_NO_TMP_DIR	6	Missing temp folder	Server config issue
UPLOAD_ERR_CANT_WRITE	7	Disk write failure	Server config issue
UPLOAD_ERR_EXTENSION	8	PHP extension blocked upload	Server config issue

3. Post-Upload Validation : Even if upload is successful (error = 0), perform these checks:

- is_uploaded_file($tmp_name): Confirms file is from a valid upload
- $_FILES['file']['size']: Re-check size against app-specific limits
- finfo_open(): Verify MIME type for security (e.g., image vs script)

10. Describe the lifecycle of a PHP session [U] / [CO2]

Solution :

1. **Creation/Initialization**
   - Begins with session_start().
   - PHP generates a unique session ID and stores it in a cookie (PHPSESSID).
   - A server-side file is created to store session data.
   - If a session ID exists, PHP resumes the session; otherwise, it creates a new one.

2. Usage (Read/Write)

- Session data is accessed via the $_SESSION array.
- Data persists across pages during the session.
- Developers can modify or remove session variables.
- PHP locks session files during access to prevent conflicts.

3. Termination/Expiration

- Sessions end via session_destroy() or timeout due to inactivity.
- session_unset() and cookie expiration are used for full logout.
- Timeout is controlled by session.gc_maxlifetime.

4. Garbage Collection

- Automatically deletes expired session files.
- Triggered probabilistically using session.gc_probability and session.gc_divisor.
- Cleans up files older than session.gc_maxlifetime.

11. Explain the role of $servername, $username, $password, and $dbname in a PHP MySQL connection. [U] / [CO3]

Solution :

- $servername: Specifies the database server's address — typically "localhost" for local setups or a remote IP/hostname for hosted databases.
- $username: The MySQL account name used for authentication — often "root" in local environments.
- $password: The password for the MySQL user — usually "" (empty) for local setups, but should be strong and secure in production.
- $dbname: The specific database name to connect to — ensures the script targets the correct data repository.

These variables are passed to functions like mysqli_connect() to establish a secure and functional link between PHP and MySQL.

PHP MySQL Connection Script

```
<?php

// Define connection variables

$servername = "localhost";     // or your server IP
$username = "root";            // default for local setup
$password = "";                // default is empty for XAMPP/WAMP
$dbname = "my_database";       // replace with your actual database name
// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);


// Check connection
if ($conn->connect_error) {
  die("Connection failed: " . $conn->connect_error);
}
echo "Connected successfully!";
?>
```

12. Explain include() and require() with examples. [U] / [CO1]

Solution :

Both include () and require () are used to insert the content of one PHP file into another, promoting code reusability and modularity.

include ()

- Generates a warning (E_WARNING) if the file is missing.
- Script continues execution even if the file is not found.
- Best for non-critical files.

  Example: include 'header.php';

require ()

- Generates a fatal error (E_COMPILE_ERROR) if the file is missing.
- Script stops execution immediately.
- Best for essential files.

  Example: require 'functions.php';

[5x5 = 25]

Part C

Essay Questions

Answer any **1** question

Each question carries **15** marks

13. Write a PHP program to find the prime numbers between 1 and 100 using a for loop. [U] / [CO1]

Solution:

```
<?php
echo "Prime numbers between 1 and 100 are: \n";
// Loop through numbers from 2 to 100
for ($num = 2; $num <= 100; $num++) {
  $isPrime = true; // Assume the number is prime initially
  // Check for divisibility from 2 up to the square root of the number
  // We only need to check up to sqrt($num) because if a number has a divisor greater than its square root,
  // it must also have a divisor smaller than its square root.
  for ($i = 2; $i * $i <= $num; $i++) {
    if ($num % $i == 0) {
      $isPrime = false; // If divisible, it's not prime
      break; // No need to check further divisors
    }
  }
  // If the number is still marked as prime (and not 1), print it
  if ($isPrime) {
    echo $num . " ";
  }
}
echo "\n";
?>
```

14. Explain the complete process of form data validation in PHP with examples. Highlight the role of isset(), empty(), and super global variables in ensuring secure data handling

[U] / [CO2]

Solution :

1.Data Retrieval

- Form inputs are accessed via superglobals:
  - $_POST for POST method
  - $_GET for GET method
- These arrays hold key-value pairs from submitted form fields.

2. Basic Validation

- isset(): Checks if a form field was submitted and is not NULL
  if (isset($_POST['username'])) { ... }
- empty(): Ensures required fields are not blank or zero.
  if (empty($_POST['username'])) { ... }

3. Data Sanitization

- Prevents malicious input:
  - trim(), stripslashes(), htmlspecialchars()
  - filter_var() for sanitizing emails, URLs, etc.

4. Validation Rules

- Apply checks based on expected input:
    - Required fields → empty()
    - Email → filter_var(..., FILTER_VALIDATE_EMAIL)
    - URL → FILTER_VALIDATE_URL
    - Numbers → is_numeric() or FILTER_VALIDATE_INT
    - Patterns → preg_match()

5. Error Handling & Feedback

- Store errors in an array and display them to users.
- Prevents re-entry of all data and improves user experience.

[15x1 = 15]